

Arquitectura y Diseño de Sistemas 2026

Lineamientos para el Diseño de APIs

Entregable 4 · Práctica de Arquitectura y Diseño de Sistemas 2026

Este documento define los lineamientos para documentar el diseño de las APIs del sistema. El formato elegido es OpenAPI 3.0, el estándar de la industria para especificación de APIs REST. El objetivo es documentar el contrato del sistema: qué expone, qué recibe, qué devuelve, y cómo maneja los errores.

No es necesario implementar todas las APIs todavía. En esta entrega se documenta el DISEÑO del contrato.

1. ¿Qué evalúa la cátedra?

La cátedra evalúa evidencia de que el equipo pensó el contrato del sistema:

- Claridad del contrato: ¿queda claro qué recibe y qué devuelve cada endpoint?
- Cohesión: ¿los endpoints tienen responsabilidades bien delimitadas?
- Manejo de errores: ¿se documentan los casos de falla, no solo el happy path?
- Coherencia con el MCD: ¿los campos del request/response coinciden con las entidades del modelo de datos?
- Convenciones REST: verbos correctos, rutas en plural, status codes semánticamente correctos.

Un buen diseño de API se puede leer y entender sin ver el código. Si alguien del equipo tuviera que implementar un cliente usando solo este documento, ¿podría hacerlo?

2. Convenciones que debe respetar el spec

Estas convenciones aplican independientemente del dominio del sistema. Deben reflejarse en el YAML.

URL base

Definir una única URL base en la sección servers. Todas las rutas son relativas a ella.

```
servers:  
- url: https://api.miapp.com/v1
```

Verbos y rutas

Verbo	Cuándo usarlo	Ejemplo
GET	Leer un recurso o lista	GET /productos, GET /usuarios/{id}
POST	Crear un recurso nuevo	POST /usuarios, POST /pedidos
PUT	Reemplazar un recurso	PUT /productos/{id}
PATCH	Modificar parcialmente	PATCH /usuarios/{id}
DELETE	Eliminar un recurso	DELETE /pedidos/{id}

- Rutas en plural y con sustantivos: /usuarios, /productos, /pedidos.
- Sin verbos en la ruta: /crearUsuario → POST /usuarios.
- Recursos anidados para relaciones: /usuarios/{id}/favoritos.

Status codes

Código	Significado	Cuándo se devuelve
200	OK	GET o PUT/PATCH exitoso que devuelve el recurso
201	Created	POST exitoso. Incluir header Location con la URL del recurso creado.
204	No Content	DELETE exitoso. Sin body.
400	Bad Request	Validación fallida: tipo incorrecto, campo faltante, formato inválido
401	Unauthorized	Sin token o token inválido/expirado
403	Forbidden	Autenticado pero sin permiso para la operación
404	Not Found	Recurso no encontrado
409	Conflict	Violación de unicidad o conflicto de estado de negocio
422	Unprocessable Entity	Estructura válida pero regla de negocio fallida
500	Internal Server Error	Error no controlado del servidor

Formato de errores

Todos los errores deben devolver el mismo formato. Definirlo una vez en components/schemas/Error y referenciar con \$ref:

```
{
  "error": {
    "code": "EMAIL_YA_REGISTRADO", // SNAKE_UPPER_CASE – identifica el error de negocio
    "message": "El email ya existe en el sistema.",
    "details": "Información adicional opcional"
  }
}
```

```
}
}
```

Campos y fechas

- Nombres de campos en camelCase: fechaInicio, userId, precioUnitario.
- Fechas siempre en ISO 8601 UTC: 2026-05-18T14:30:00Z — usar format: date-time en el spec.
- Listas paginadas con estructura estándar: { data: [...], pagination: { page, limit, total, totalPages } }.

3. Ejemplo completo anotado

El siguiente spec cubre un módulo de Usuarios y Favoritos con todos los lineamientos aplicados. Los comentarios (líneas en verde) explican cada decisión. Adaptar al dominio del grupo.

```
openapi: 3.0.3
info:
  title: API de Usuarios          # Nombre visible en Swagger UI
  version: 1.0.0                 # Versión del contrato (no del sistema)

servers:
  - url: https://api.miapp.com/v1 # URL base – todas las rutas son relativas a esta

paths:

  # — MÓDULO: USUARIOS —————

  /usuarios/registro:
    post:
      summary: Registrar un nuevo usuario
      tags: [Usuarios]           # Agrupa endpoints en la UI – usar un tag por módulo
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, password] # Listar los campos obligatorios
              properties:
                email:
                  type: string
                  format: email           # Validación semántica automática
                  example: usuario@mail.com
                password:
                  type: string
                  minLength: 8            # Restricciones de negocio van acá
      responses:
        '201':                     # Usar comillas en los códigos HTTP
          description: Usuario creado exitosamente
          headers:
            Location:               # Buena práctica: devolver URL del recurso
              schema: { type: string }
              example: /usuarios/abc-123
        '409':                     # Documentar los errores esperables
          description: El email ya está registrado
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Error' # Reutilizar – no repetir

  /usuarios/login:
    post:
```

```

summary: Iniciar sesión y obtener JWT
tags: [Usuarios]
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        required: [email, password]
        properties:
          email:
            type: string
            format: email
          password:
            type: string
responses:
  '200':
    description: Login exitoso
    content:
      application/json:
        schema:
          type: object
          properties:
            token:
              type: string
              description: JWT – usar en Authorization: Bearer <token>
  '401':
    description: Credenciales inválidas
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'

```

— MÓDULO: FAVORITOS —

```

/usuarios/{id}/favoritos:
  get:
    summary: Listar favoritos del usuario
    tags: [Favoritos]
    security:
      - bearerAuth: []          # Endpoint protegido – requiere JWT
    parameters:
      - in: path                # 'path' para {id}, 'query' para ?param=valor
        name: id
        required: true
        schema:
          type: string
          format: uuid
      - in: query               # Paginación estándar
        name: page
        schema: { type: integer, default: 1 }
      - in: query
        name: limit
        schema: { type: integer, default: 20, maximum: 100 }
    responses:
      '200':
        description: Lista de productos favoritos
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:
                    $ref: '#/components/schemas/Producto'
                pagination:
                  $ref: '#/components/schemas/Pagination'
      '401':
        description: Token no enviado o expirado
        content:

```

```

        application/json:
          schema: { $ref: '#/components/schemas/Error' }
      '403':
        description: No puede ver los favoritos de otro usuario
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }

post:
  summary: Agregar un producto a favoritos
  tags: [Favoritos]
  security:
    - bearerAuth: []
  parameters:
    - in: path
      name: id
      required: true
      schema: { type: string, format: uuid }
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          required: [productoId]
          properties:
            productoId:
              type: string
              format: uuid
  responses:
    '201': { description: Favorito agregado }
    '409':
      description: El producto ya está en favoritos
      content:
        application/json:
          schema: { $ref: '#/components/schemas/Error' }

/usuarios/{id}/favoritos/{productoId}:
  delete:
    summary: Quitar un producto de favoritos
    tags: [Favoritos]
    security:
      - bearerAuth: []
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string, format: uuid }
      - in: path
        name: productoId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '204': { description: Eliminado – sin body }
      '404':
        description: El favorito no existe
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }

# — COMPONENTES REUTILIZABLES —

components:
  securitySchemes:
    bearerAuth: # Nombre referenciado con 'security: - bearerAuth: []'
      type: http
      scheme: bearer
      bearerFormat: JWT

  schemas:
```

```

Error:                                # Definir una vez, reutilizar con $ref en todos los
endpoints
  type: object
  properties:
    error:
      type: object
      properties:
        code:
          type: string
          example: EMAIL_YA_REGISTRADO # SNAKE_UPPER_CASE – identifica el error de
negocio
        message:
          type: string
          example: El email ya existe en el sistema
        details:
          type: string
          description: Información adicional opcional

  Pagination:
    type: object
    properties:
      page:      { type: integer }
      limit:     { type: integer }
      total:     { type: integer }
      totalPages: { type: integer }

Producto:                             # Esquema de entidad – adaptar al modelo de datos del
grupo
  type: object
  properties:
    id:      { type: string, format: uuid }
    nombre:  { type: string }
    precio:  { type: number, format: float }
    creadoEn: { type: string, format: date-time } # ISO 8601 UTC

```

Este spec es un punto de partida. Adaptar al dominio del grupo: renombrar entidades, agregar endpoints, ajustar schemas al modelo de datos de la Entrega 3.

4. Cómo visualizar el spec

Swagger UI convierte el YAML en una interfaz web interactiva. La opción más rápida es Swagger Editor online:

- Ir a editor.swagger.io
- Pegar el contenido del archivo .yaml en el panel izquierdo.
- El panel derecho muestra la documentación renderizada y los errores de validación.

Para la entrega: subir el archivo .yaml al repositorio e incluir un screenshot de Swagger Editor con el spec validado.

Otras opciones si quieren integrarlo al proyecto:

Opción	Cómo usarlo	Cuándo conviene
Swagger Editor online	editor.swagger.io — pegar el YAML	Siempre: sin instalación, validación inmediata

Opción	Cómo usarlo	Cuándo conviene
Stoplight Studio	app.stoplight.io — editor visual	Para escribir el spec con autocompletado
swagger-ui-express (Node)	npm install swagger-ui-express	Si el backend es Express y quieren servir la doc
springdoc-openapi (Java)	Dependencia Maven/Gradle	Si el backend es Spring Boot

5. Checklist de entrega

Verificar que el spec cumple con todo lo siguiente antes de entregar:

Estructura general:

- ☐ ¿El archivo .yaml está en el repositorio?
- ☐ ¿Valida sin errores en editor.swagger.io?
- ☐ ¿Está definida la URL base en servers?
- ☐ ¿Los endpoints están agrupados por módulo con tags?

Por cada endpoint:

- ☐ ¿Tiene summary descriptivo en términos de negocio?
- ☐ ¿Indica si requiere autenticación (security: - bearerAuth: []) o es público?
- ☐ ¿Están documentados los path parameters y query parameters?
- ☐ ¿El requestBody tiene los campos con sus tipos y cuáles son required?
- ☐ ¿La response exitosa tiene el schema correcto?
- ☐ ¿Se documentan al menos los errores más esperables (401, 404, 409...)?
- ☐ ¿Los status codes son semánticamente correctos (no 200 para todo)?

Schemas y coherencia:

- ☐ ¿Los errores usan \$ref a un esquema Error reutilizable?
- ☐ ¿Los campos coinciden con las entidades del modelo de datos de la Entrega 3?
- ☐ ¿Los nombres de campos están en camelCase?
- ☐ ¿Las fechas usan format: date-time (ISO 8601 UTC)?
- ☐ ¿Los errores tienen code en SNAKE_UPPER_CASE?

Pregunta de autoevaluación: si alguien del equipo tuviera que implementar un cliente para su API usando solo este spec, ¿podría hacerlo? Eso es lo que busca la cátedra.